

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA1014	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	Total Marks:	25
CO1 Statement:	Apply advanced methodologies and agile project management techniques to manage software projects effectively		
Student Name:	Arunprasath R	Reg. No.:	192524077
Date:	05/10/2026	Duration:	60 minutes

Code of Conduct

I Arunprasath R (192524077) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: R.Arunprasath

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. Understand the Problem Statement

- Explain the problem in your own words.
- Identify the objectives of the proposed system.
- State the scope of the problem.

2. Identify Key Stakeholders and Challenges

- Identify all the stakeholders involved in the system.
- Explain the roles and responsibilities of each stakeholder.
- Discuss the major challenges and issues faced by the stakeholders.

3. Analyze the Current Approach

- Describe the existing system or current process.
- Identify its strengths and limitations.
- Analyze the problems or gaps that need to be addressed.

4. Propose a Suitable Solution Using Software Engineering Principles

- Design a software-based solution for the given problem.
- Describe the major functional modules of the proposed system.
- Identify the functional and non-functional requirements.
- Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.

5. Justify the Chosen Methodology/Framework

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).

- Justify why the selected methodology/framework is suitable for the proposed system.
 - Explain how it supports successful project development and maintenance.
- 6. Discuss Expected Outcomes and Limitations**
- Describe the expected benefits of the proposed solution.
 - Explain how the solution addresses the identified challenges.
 - Discuss potential risks, implementation challenges, and limitations.
 - Suggest possible future enhancements.

QUESTION:

2. Intelligent Academic Timetable Conflict Resolution Platform

Industry Context:

Universities must schedule hundreds of courses, faculty members, classrooms, laboratories, and student batches. Manual timetable preparation can create conflicts and inefficient room utilization.

Problem Statement:

Design and develop an intelligent timetable platform that automatically detects scheduling conflicts and generates feasible timetables based on faculty availability, classroom capacity, laboratory requirements, course combinations, and student batches.

Objectives:

- Manage academic scheduling constraints.
- Detect faculty conflicts.
- Detect classroom conflicts.
- Handle laboratory requirements.
- Optimize room utilization.
- Support timetable modifications.

Expected Outcomes:

- Conflict-free timetables.
- Reduced administrative effort.
- Better classroom utilization.
- Faster timetable generation.
- Improved academic scheduling.

ANSWER:

1. Understand the Problem Statement:

1.1 Problem Explanation:

In universities, preparing an academic timetable is a complex task because many courses, faculty members, classrooms, laboratories, and student batches must be scheduled within limited time slots.

Currently, timetable preparation is often performed manually using spreadsheets or other basic tools. This can result in several problems such as faculty clashes, classroom clashes, laboratory conflicts, and overlapping classes for the same student batch.

The proposed Intelligent Academic Timetable Conflict Resolution Platform is a software system that automatically analyzes scheduling constraints, detects conflicts, and generates a feasible timetable. The system considers factors such as faculty availability, classroom capacity, laboratory requirements, course combinations, and student batch availability.

The platform aims to reduce manual effort and produce a more efficient and conflict-free academic timetable.

1.2 Objectives of the Proposed System:

The main objectives are:

- To automatically generate academic timetables.
- To detect faculty scheduling conflicts.
- To detect classroom and laboratory conflicts.
- To consider faculty availability while scheduling.
- To assign classrooms based on capacity and requirements.
- To handle laboratory-specific requirements.
- To avoid overlapping classes for the same student batch.
- To improve classroom and laboratory utilization.
- To allow administrators to modify generated timetables.
- To reduce the time and effort required for timetable preparation.

1.3 Scope of the Problem:

The proposed system can be used by universities, colleges, and academic institutions for timetable planning.

The scope includes:

- Course and subject management.

- Faculty management.
- Classroom and laboratory management.
- Student batch management.
- Faculty availability management.
- Scheduling of courses into available time slots.
- Automatic conflict detection.
- Timetable generation.
- Timetable modification.
- Room allocation and utilization.
- Viewing and exporting timetables.

The system mainly focuses on academic timetable scheduling and conflict resolution. It does not directly manage other university activities such as examination scheduling, student attendance, or payroll.

2. Identify Key Stakeholders and Challenges:

2.1 Key Stakeholders:

The major stakeholders involved in the system are:

Stakeholder	Role and Responsibilities
Academic Administrator	Manages the overall timetable and scheduling process.
Department Coordinator	Provides course, faculty, batch, and scheduling information.
Faculty Members	Provide availability and view their assigned timetable.
Students	View their class timetable and identify their scheduled courses.
IT/System Administrator	Maintains the software, users, database, and system security.
University Management	Monitors scheduling efficiency and resource utilization.

Academic Administrator - The academic administrator is responsible for creating and managing the overall timetable. They enter scheduling requirements, generate timetables, identify conflicts, and approve the final schedule.

Department Coordinator - The department coordinator provides information about courses, faculty members, student batches, classrooms, laboratories, and required time slots.

Faculty Members - Faculty members provide their available and unavailable time slots. They can view their assigned courses and timetable.

Students - Students use the platform to view their timetable. The system ensures that subjects belonging to the same batch do not overlap.

IT/System Administrator - The system administrator manages user accounts, system configuration, database maintenance, backups, and security.

University Management - Management can use timetable information to analyze classroom utilization, faculty workload, and overall scheduling efficiency.

2.2 Major Challenges Faced by Stakeholders:

The major challenges include:

1. **Faculty conflicts**
The same faculty member may accidentally be assigned to two classes at the same time.
2. **Classroom conflicts**
A classroom may be assigned to multiple courses during the same time slot.
3. **Student batch conflicts**
Students may have two or more classes scheduled at the same time.
4. **Laboratory availability**
Laboratories may have limited capacity and specific equipment requirements.
5. **Faculty availability**
Some faculty members may not be available during certain time slots.
6. **Room capacity mismatch**
A classroom may not have sufficient capacity for a particular student batch.
7. **Frequent modifications**
Changes in faculty availability or room allocation may require timetable modifications.
8. **Large number of constraints**
Managing hundreds of courses and resources manually becomes difficult.
9. **Time consumption**
Manual timetable generation can take considerable administrative time.

3. Analyze the Current Approach:

3.1 Existing System:

In many institutions, timetable preparation is performed using:

- Manual scheduling.
- Spreadsheets such as Excel.
- Basic timetable software.
- Department-level scheduling.

The administrator generally collects information about courses, faculty, classrooms, and student batches and manually assigns subjects to different time slots.

The timetable is then checked manually for conflicts and modified whenever problems are found.

Existing Process: Collect Information → Assign Time Slots → Assign Faculty → Assign Rooms → Check Conflicts → Modify Timetable → Finalize Timetable

3.2 Strengths of the Existing Approach:

The current approach has some advantages:

- Easy to understand.
- Does not require sophisticated software.
- Flexible for small departments.
- Administrators can directly make changes.
- Spreadsheet-based systems are inexpensive.
- Existing staff may already be familiar with the process.

3.3 Limitations of the Existing Approach:

However, manual scheduling has several limitations:

- High possibility of human errors.
- Conflict detection is time-consuming.
- Difficult to manage large numbers of courses.
- Faculty clashes may be overlooked.
- Classroom utilization may be inefficient.
- Laboratory requirements are difficult to manage.
- Changes require repeated manual checking.
- No automatic optimization.
- Difficult to handle multiple constraints simultaneously.
- Maintaining different versions of timetables can create confusion.

3.4 Problems and Gaps to be Addressed:

The major gaps in the current system are: Manual Work → High Errors → Difficult Conflict Detection → Poor Resource Utilization → Time-Consuming Modifications

Therefore, there is a need for an intelligent software platform that can automatically analyze constraints and generate a feasible timetable.

4. Proposed Solution Using Software Engineering Principles:

4.1 Proposed Software Solution:

The proposed system is an Intelligent Academic Timetable Conflict Resolution Platform.

The platform collects information about:

- Courses
- Faculty
- Student batches
- Classrooms
- Laboratories
- Time slots
- Faculty availability
- Room capacity
- Laboratory requirements

The scheduling engine analyzes these constraints and generates a timetable that avoids major conflicts.

A simplified workflow is : Input Data → Constraint Analysis → Conflict Detection → Timetable Generation → Optimization → Validation → Final Timetable

If a conflict occurs, the system identifies the conflict and suggests an alternative time slot or room.

4.2 Major Functional Modules:

Module 1: User Management

This module manages different users such as:

- Administrators
- Department coordinators
- Faculty members
- Students

It provides role-based access to the system.

Module 2: Course Management

The administrator can:

- Add courses.
- Modify course details.
- Assign faculty.
- Define course duration.
- Define required weekly sessions.
- Specify laboratory requirements.

Module 3: Faculty Management

This module manages:

- Faculty details.
- Assigned courses.
- Faculty availability.
- Faculty workload.
- Preferred or unavailable time slots.

Module 4: Classroom and Laboratory Management

The system stores:

- Room number.
- Room type.
- Capacity.
- Available time slots.
- Laboratory facilities.
- Equipment requirements.

The system ensures that a room is not assigned to two classes simultaneously.

Module 5: Student Batch Management

This module manages student batches and their courses.

For example:

- B.Tech AI & Data Science – Year 1
- B.Tech CSE – Year 2
- B.Tech ECE – Year 3

The system prevents overlapping courses for the same batch.

Module 6: Constraint Management

This is one of the major modules of the system.

It handles constraints such as:

- Faculty availability.
- Classroom capacity.

- Laboratory availability.
- Student batch availability.
- Course combinations.
- Time-slot restrictions.

Module 7: Timetable Generation Engine

The scheduling engine automatically assigns:

Course + Faculty + Room + Time Slot + Student Batch

while attempting to satisfy the defined constraints.

Module 8: Conflict Detection and Resolution

The system detects:

- Faculty conflicts.
- Classroom conflicts.
- Laboratory conflicts.
- Student batch conflicts.
- Capacity conflicts.
- Availability conflicts.

It then suggests alternative assignments where possible.

Module 9: Timetable Modification

Administrators can modify the generated timetable.

For example:

Faculty unavailable → Select another available slot → Check room availability → Validate conflicts → Update timetable

Module 10: Reports and Timetable Viewing

The system can generate:

- Faculty timetable.
- Student timetable.
- Classroom timetable.
- Department timetable.
- Room utilization reports.

- Conflict reports.

4.3 Functional Requirements:

The system should:

1. Allow administrators to log in securely.
2. Allow users to manage courses.
3. Allow faculty information to be added and modified.
4. Allow classroom and laboratory information to be maintained.
5. Allow student batches to be managed.
6. Store faculty availability.
7. Store room capacity and laboratory requirements.
8. Generate timetables automatically.
9. Detect scheduling conflicts.
10. Provide conflict resolution suggestions.
11. Allow timetable modifications.
12. Display timetables based on user roles.
13. Generate timetable reports.
14. Store and retrieve timetable data.

4.4 Non-Functional Requirements:

Performance : The system should generate timetables within a reasonable amount of time even when many courses and resources are involved.

Usability : The interface should be simple and easy for administrators, faculty members, and students to use.

Reliability : The system should consistently generate valid timetables and prevent major scheduling conflicts.

Scalability : The platform should support an increasing number of: Courses , Faculty , Classrooms , Laboratories , Student batches.

Security : Only authorized users should be allowed to modify timetable information.

Maintainability : The system should be designed in modules so that individual components can be updated without affecting the entire application.

Availability : The platform should be available to users whenever timetable information needs to be accessed.

4.5 Application of Software Engineering Principles:

- 1. Modularity** - The system is divided into independent modules such as Faculty Management, Course Management, Room Management, Scheduling, and Conflict Detection. This makes development and maintenance easier.
- 2. Scalability** - The system can be designed to support additional departments, courses, faculty members, and classrooms without major changes to the architecture.
- 3. Maintainability** - A modular code structure makes it easier to identify errors, implement updates, and add new features.
- 4. Reliability** - Before finalizing a timetable, the system validates faculty, room, laboratory, and student batch constraints.
- 5. Usability** - A simple dashboard allows users to enter information, generate timetables, view conflicts, and make modifications easily.
- 6. Security** - Role-based authentication ensures that only authorized users can modify important scheduling data.
- 7. Reusability** - Common components such as conflict checking, room validation, and user authentication can be reused throughout the application.

5. Justify the Chosen Methodology/Framework:

5.1 Selected SDLC Model: Agile Scrum

For this project, the **Agile Scrum methodology** is suitable.

Agile Scrum develops the software incrementally through short development cycles called **Sprints**.

The system can be divided into multiple sprints.

Example Sprint Plan

Sprint	Major Activities
Sprint 1	Requirement analysis and user management
Sprint 2	Course and faculty management
Sprint 3	Classroom and laboratory management
Sprint 4	Constraint and conflict detection
Sprint 5	Timetable generation engine
Sprint 6	Modification, reports, testing and deployment

5.2 Why Agile Scrum is Suitable:

Agile is suitable because timetable requirements can change frequently.

For example, the university may later request:

- Additional constraints.
- New departments.
- New types of classrooms.
- Different scheduling rules.
- Faculty preferences.
- New reports.

Agile allows these changes to be incorporated during later sprints.

Advantages of Agile for This Project:

- Supports changing requirements.
- Provides regular working software.
- Allows continuous testing.
- Enables frequent feedback from administrators.
- Reduces development risk.
- Makes debugging easier.
- Supports incremental development.
- Helps improve the system based on user feedback.

Therefore, Agile Scrum provides flexibility and continuous improvement, making it appropriate for developing the timetable platform.

6. Expected Outcomes and Limitations:

6.1 Expected Benefits:

The proposed system is expected to provide the following benefits:

- 1. Conflict-Free Timetables** - The system automatically checks faculty, classroom, laboratory, and student batch conflicts.
- 2. Reduced Administrative Effort** - Automatic timetable generation reduces the amount of manual work.
- 3. Faster Timetable Generation** - The scheduling engine can analyze multiple constraints faster than manual scheduling.

4. Better Classroom Utilization - The system can assign suitable rooms based on capacity and availability.

5. Efficient Laboratory Scheduling - Laboratories can be assigned according to equipment and capacity requirements.

6. Easy Timetable Modification - Administrators can modify schedules and immediately validate the changes.

7. Improved Academic Planning - Faculty and students receive organized and accessible timetables.

8. Reduced Human Errors - Automated validation reduces common mistakes in manual timetable preparation.

6.2 How the Solution Addresses the Challenges:

Existing Challenge	Proposed Solution
Faculty clashes	Automatic faculty conflict detection
Classroom clashes	Room availability validation
Student batch overlap	Batch-wise scheduling validation
Laboratory conflicts	Laboratory-specific constraints
Room capacity issues	Capacity-based room allocation
Faculty availability	Availability-based scheduling
Manual work	Automated timetable generation
Frequent changes	Timetable modification module
Poor resource utilization	Room utilization analysis
Human errors	Automatic validation

6.3 Potential Risks and Implementation Challenges:

1. Large Number of Constraints - Universities may have hundreds of courses and many scheduling rules. Handling all constraints efficiently can be challenging.

2. Complex Scheduling - Some courses may require multiple sessions, specific laboratories, or special faculty availability.

3. Changing Requirements - University scheduling policies may change frequently.

4. Data Accuracy - Incorrect faculty availability, room capacity, or course information can result in an incorrect timetable.

5. Algorithm Complexity - Generating an optimal timetable for a large institution can require significant computational resources.

6. User Adoption - Staff members familiar with manual methods may initially find the new system difficult to adopt.

7. System Dependency - If the platform becomes unavailable, administrators may temporarily face difficulties accessing or modifying schedules.

6.4 Limitations:

The proposed system may have some limitations:

- It may not always produce the mathematically optimal timetable.
- Highly complex university scheduling rules may require additional configuration.
- The quality of the timetable depends on the accuracy of input data.
- Large scheduling problems may require more processing time.
- Human approval may still be required before publishing the final timetable.
- Sudden real-world changes may require manual intervention.

6.5 Future Enhancements:

The system can be enhanced in the future by adding:

1. AI-based timetable optimization for more efficient scheduling.
2. Predictive scheduling based on historical timetable data.
3. Faculty preference analysis for better workload distribution.
4. Mobile application for students and faculty.
5. Real-time notifications when timetable changes occur.
6. Integration with university ERP/LMS systems.
7. Automated attendance system integration.
8. Examination timetable scheduling.
9. Advanced room utilization analytics.
10. What-if scheduling, allowing administrators to compare multiple timetable possibilities.
11. Cloud deployment for access across departments.
12. Automatic rescheduling when a faculty member or classroom becomes unavailable.

Conclusion:

The Intelligent Academic Timetable Conflict Resolution Platform provides a software-based solution to the difficulties associated with manual academic timetable preparation. The system manages courses,

faculty members, classrooms, laboratories, student batches, and scheduling constraints in an organized manner.

By automatically detecting conflicts and generating feasible timetables, the system can reduce administrative effort, minimize human errors, improve classroom utilization, and speed up timetable preparation.

The use of modularity, scalability, maintainability, reliability, usability, and security ensures that the proposed platform follows important software engineering principles. The Agile Scrum methodology further supports incremental development, continuous testing, and adaptation to changing university requirements.

Overall, the proposed platform can provide a more efficient, reliable, flexible, and user-friendly approach to academic timetable management.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity,	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.

	scalability, maintainability, security, etc.).			
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25